Reproducible Workflows and GitHub

INF2167, Week 2

Inessa De Angelis

May 14, 2026

Announcements

- No office hours on Monday (May 18) because of Victoria Day. Please email me to schedule a time if you would like to meet next week
- Pairs for the group project have been set
- We will have an opportunity to meet and work with our group project partners during today's lab

Working without Quercus

Our alternative class pages while Quercus is still coming back online are:

- Class OneDrive folder for the syllabus, assignment instructions/rubrics, and slides (must be logged in using your UofT credentials to view)
- Class GitHub Repository for slides, live coding examples, and in-class labs
- Email for announcements and submitting in-class labs

I appreciate your flexibility and patience as we navigate these uncertain circumstance together

Reminder: Course Policy on AI

- **R code:** If you use AI for the development and/or de-bugging of code, the use of such tools MUST be explicitly acknowledged and cited in submitted work. All relevant prompts and responses should be included in a text file in your repo and thoroughly commented in your scripts. Failure to do so may result in penalties
- **Written work:** The use of generative AI tools or apps for written assignments in this course, including the final project proposal and paper is PROHIBITED. Representing one's own idea, or expression of an idea that was AI-generated may be considered an academic offense in this course. You may not copy or paraphrase from any generative AI applications, for the purpose of completing written course assignments

Reminder: Course communications

- Accommodation, extension, re-grade requests need to be made using the course notifications/accommodations form before the due date/class

Objectives

- Differentiate between question/theory-driven and data-driven research approaches
- Think about how to set up reproducible research workflows
- Start to apply workflow thinking at both the project level and within R scripts
- Understand the role of data simulation in research workflows
- Meet your group project partner and start thinking about your topic and how to conceptualize your final project

Big picture

In today's class, we are going to think about research as an end-to-end system:

- How do we turn ideas into research questions?
- How do we move from data -> analysis -> results?
- How do we make this process reproducible and even automated?

The end goal is to think about and treat research as a workflow, not isolated steps

Data science workflows

- A workflow is a clearly defined series of steps used to complete a research project or analysis (De Angelis, 2026)
- In practice, workflows refer both to:
  - the larger stages of a research project from start to finish
  - and the processes within individual R scripts

Data science workflows (continued)

- A common data science workflow is:
  - Plan -> Simulate -> Acquire -> Explore -> Share (Alexander, 2023)
- Although workflows are often presented linearly, research is an iterative practice
  - we move back and forth between stages
  - we refine questions based on data
  - we adjust data collection based on feasibility

Stages of the data science workflow

- **Plan:** Develop research questions, theory, and identify data needs
- **Simulate:** Create a dataset to test ideas, assumptions, and workflows
- **Acquire:** Collect or obtain data from surveys, APIs, experiments, web scraping, etc.
- **Explore:** Clean, visualize, summarize, and analyze data
- **Share:** Communicate our findings in a reproducible way through papers, reports, and visualizations

Plan

Planning is not just “coming up with an idea” - it includes:

- defining research questions
- reviewing theory/literature
- determining feasibility
- identifying possible data sources
- and lots of refining the scope

How do you start designing a research project?

- When starting to design a research project, how do you go about it?

Designing a research project

- Broadly, there are two ways to go about research:
  - Data-first
  - Questions/theory-first
- But it is not a binary process - there is often iteration between data and questions/theory

Data-first

- When focusing on data-first, the main issue is figuring out what research questions can be answered with the available data
- But we also need to make sure the questions we ask are relevant and whether there is theory that can support and explain our analyses
- Having data now is good, but it is important to consider whether that data will be available again in the future (to you or other researchers)

Questions/theory-first

- This is the practice of establishing a clear and focused research question supported by theory and previously published literature, before collecting data
- This approach can help prevent common research problems, such as pursuing unanswerable questions or collecting irrelevant data
- The “FINER framework” is used in medicine to help guide the development of research questions. It recommends asking questions that are: Feasible, Interesting, Novel, Ethical, and Relevant (Alexander, 2023)

Example: Data-first

- Throughout 2024, I collected YouTube video information and comment data from all federal Canadian Members of Parliament (MPs) active on the platform with the intention of generally studying political communication and online harassment
- I chose to collect this data because I knew that YouTube is understudied in the Canadian political communication literature and any of the papers I wrote using this data would be theoretically relevant and interesting
- Now, I've used this entire dataset or subsets of it for a handful of papers (either under review at a journal or in progress)

Example: Questions/theory-first

- My masters' thesis research examined the severity of harassment received by Canadian women politicians when tweeting about climate change (De Angelis, 2024)
- I first became interested in this topic after working with former Cabinet Minister and MP, Catherine McKenna and seeing firsthand her experiences with harassment
- I then reviewed the academic literature and noticed that very few studies examined the intersection between climate change and gender-based harassment faced by women politicians
- Therefore, we can say that this project was originally driven by my lived-experience and question/theory-first, but then iterated through data and questions/theory as it progressed

Types of questions

- One way to figure out what type of research questions we want to ask is to figure out if we are interested in doing descriptive, predictive, inferential, or causal analysis
  - **Descriptive:** What does x look like?
  - **Predictive:** What will happen to x?
  - **Inferential:** How can we explain x?
  - **Causal:** What impact does x have on y?

Data and types of questions

- We need to be careful when conceptualizing our research projects about selection and measurement bias
- Selection bias happens when our results depend on who is in the sample, rather than reflecting the broader population
- Example: If we want to study political opinions in Canada but only use data from Twitter/X, we exclude people who do not use the platform. Those excluded groups may systematically differ in age, ideology, or level of political engagement, which could make our findings unrepresentative

Data and types of questions (continued)

- Measurement bias happens when our results are affected by how the data are collected, defined, or measured
- Example: If we measure online harassment using only keyword filters (for example, swear words), we may miss more subtle forms of harassment such as subtle sexism or racism. This could lead to an underestimation of the forms and impacts of online harassment
- We will learn more about selection and measurement bias during Week 4 (May 28th)

Simulating data

- Why might we want to simulate data as part of our research workflow?

Simulating data

We simulate data to:

- Get a sense of what real data might look like prior to collection (helps us anticipate missing data or irregularities)
- Test assumptions underlying our analyses
- Debug code and research workflows before working with our real data
- Work with data that are unavailable, sensitive, or cannot be publicly shared

Best practices for simulating data

- Incorporate domain knowledge to generate more realistic datasets
- Clearly document assumptions used in the simulation process
- Use `set.seed()` to ensure reproducibility
- Remember that simulated data are useful for testing and learning, but may not reflect real-world complexity

Example: simulating data

- `set.seed(16)` sets the random seed so the simulated data can be reproduced exactly each time the code is run
- This is important for reproducibility
- You can choose *any* integer to put inside `set.seed()`
- The specific value does not matter, but changing the number will create a different random sample

```
set.seed(16)

simulated_data <- tibble(
  comment_id = 1:1000,
  severity_of_harassment = sample(
    x = 1:7,
    size = 1000,
    replace = TRUE,
    prob = c(0.05, 0.10, 0.425, 0.20, 0.15, 0.05, 0.025)))
```

Example: simulating data (continued)

- We create a new object called `simulated_data`
- `tibble()` creates a tidy data frame where everything inside the parentheses becomes columns in the dataset
- `comment_id = 1:1000` creates a column called `comment_id`
- `1:1000` generates a sequence from 1 to 1,000, acting as a unique identifier for each simulated observation (each "comment")

```
 1 set.seed(16)
 2
 3 simulated_data <- tibble(
 4   comment_id = 1:1000,
 5   severity_of_harassment = sample(
 6     x = 1:7,
 7     size = 1000,
 8     replace = TRUE,
 9     prob = c(0.05, 0.10, 0.425, 0.20, 0.15, 0.05, 0.025)))
10
```

Example: simulating data (continued)

- `severity_of_harassment` = `sample(...)` creates a column called `severity_of_harassment`
- Values are generated randomly using the `sample()` function
- `x = 1:7` defines the possible values that can be assigned
- The `severity of harassment` scores can take values from 1 to 7 (1 = positive and 7 = hate speech)

```
1  set.seed(16)
2
3  simulated_data <- tibble(
4    comment_id = 1:1000,
5    severity_of_harassment = sample(
6      x = 1:7,
7      size = 1000,
8      replace = TRUE,
9      prob = c(0.05, 0.10, 0.425, 0.20, 0.15, 0.05, 0.025)))
```

Example: simulating data (continued)

- `size = 1000` tells R to generate 1,000 values (matching the number of rows in the dataset)
- `replace = TRUE` allows values to be repeated
- Without this, each number could only appear once, which would make it impossible to generate 1,000 observations from only 7 categories
- `prob = ...` sets the probability of selecting each value from 1 to 7

```
set.seed(16)

simulated_data <- tibble(
  comment_id = 1:1000,
  severity_of_harassment = sample(
    x = 1:7,
    size = 1000,
    replace = TRUE,
    prob = c(0.05, 0.10, 0.425, 0.20, 0.15, 0.05, 0.025)))
```

Example: simulating data (continued)

```
1  head(simulated_data)
```

`head()` shows us what the first 6 rows of our simulated dataset look like

```
# A tibble: 6 × 2
  comment_id severity_of_harassment
       <int>                  <int>
1          1                      5
2          2                      3
3          3                      4
4          4                      3
5          5                      2
6          6                      3
```

Example: simulating data (continued)

- We can now perform the same types of exploratory analysis that we did last week using functions such as `filter()` and `count()`

```
simulated_data |>
  group_by(severity_of_harassment) |>
  count() |>
  ungroup() |>
  mutate(percentage = round(n / sum(n) * 100, 2))
```

Example: simulating data (continued)

- We can see that more nuanced forms of harassment (3, 4, 5) are the most prevalent
- These probabilities are determined based on my domain knowledge and results from a previous study (De Angelis, 2024)

```
# A tibble: 7 × 3
  severity_of_harassment     n percentage
                   <int> <int>      <dbl>
1                      1    44        4.4
2                      2   115       11.5
3                      3   421       42.1
4                      4   179       17.9
5                      5   164       16.4
6                      6    50        5
7                      7    27        2.7
```

Other useful functions for simulating data

Distributions may also be useful:

- `rnorm()` for continuous data (age, height, test scores)
- `rbinom()` for binary data (yes/no, success/failure)
- `runif()` for uniform ranges (wait times, lottery draws)

You may wish to consult Chapter 2 of *Telling Stories with Data* for examples of these functions in use

Common challenges in research design

- Asking questions that we cannot reasonably answer with the data we have
- Collecting data without a clear research question (a general idea is fine though)
- Ignoring bias in data collection
- Not documenting data collection, cleaning, and methodological choices

Estimands

- An estimand is a precise description of the effect or quantity we want to learn from the data
- In other words, it defines exactly what we are trying to estimate or measure
- For example, if we are studying whether exposure to online harassment affects politicians' posting behaviour, our estimand might be: "The average change in the number of tweets posted after politicians receive harassment, compared to if they had not received harassment"
- We will typically want to define this in the introduction of our project proposal or paper

Share: Reproducible research workflows

- The final stage of the data science workflow is sharing our work in a reproducible way
- Reproducibility means that future you or someone else can understand and rerun your analysis
- Reproducible workflows include:
  - organized project structures
  - clearly documented code
  - version control
  - and properly cited sources and software

Components of a paper

- Academic papers are one of the main ways researchers share their work
- In our papers, we will typically want to have: Title, Abstract, Introduction, Literature Review/Theory (or Previous Research), Data and Methods, Model, Results and Discussion, Conclusion, References
- Not all papers have all of these components and sometimes they are called similar, but different things

Components of a paper (continued)

- When writing our papers using Quarto files, we need to have a separate bibliography file
- These are called BibTeX files and they help automate citation management
- You **must** cite R, R packages, and datasets in addition to scholarly literature (this is expected for all components of the final project)

BibTeX files (.bib)

- A `.bib` file is a plain-text database of your references
- Each entry has a type (for example, @ article), a unique citation key, and metadata (author, year, title)
- We can include references by specifying a BibTeX file in the top matter of our .qmd file and then calling it within the text
- You can export citations in BibTex format from Zotero or Google Scholar

Example BibTeX file entry

```
@Manual{r,
  title = {{R: A Language and Environment for Statistical Computing}},
  author = {{R Core Team}},
  organization = {R Foundation for Statistical Computing},
  address = {Vienna, Austria},
  year = {2025},
  url = {https://www.R-project.org/},
  }
```

Project-level workflow structures

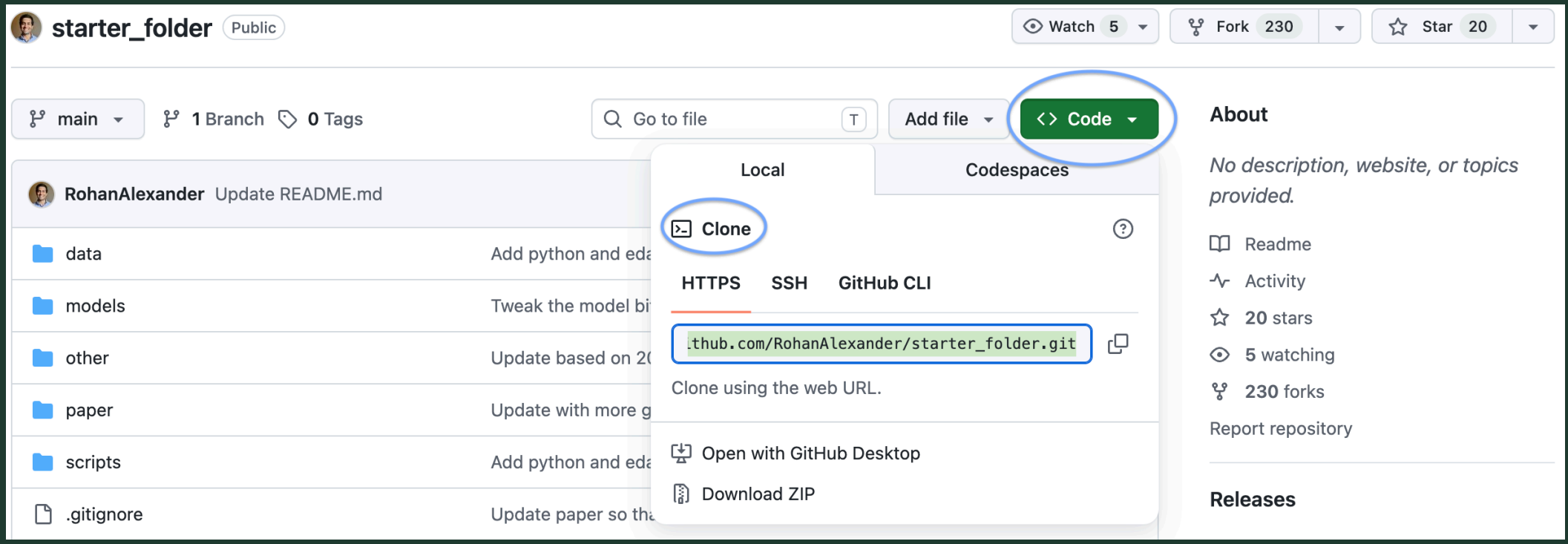
- In this class, our workflows will have two components: our local RStudio component and then our remote version on GitHub
- This is called "**version control**" and it is a system that records changes to files or code over time
- This is to help ensure reproducibility, improve workflow efficiency by being systematic, facilitate collaboration, and save a backup of our code and data in case of any accidents (i.e., you spill water all over your computer!)
- To link the two, we'll need to make sure GitHub is linked to our RStudio and the correct permissions have been set to push and pull commits. We'll talk more about how to do this in a moment

R Projects and file structure

```
project/
├── data/
|   ├── raw/
|   ├── simulated/
|   └── analysis/
├── scripts/
|   ├── 01-download_data.R
|   ├── 02-clean_data.R
|   ├── 03-eda.R
|   └── 04-model.R
├── model/
|   └── model_output.rds
├── paper/
|   ├── paper.qmd
|   └── references.bib
├── other/
|   ├── literature/
|   ├── datasheet/
|   └── sketches/
├── README.md
├── LICENSE.md
└── project.Rproj
```

R Projects and file structure (continued)

- Different projects may require variations of this file structure, but it is a good idea to work with a similar structure
- You are welcome to copy Prof. Alexander's starter_folder repo to set up your own repo (make sure to clone, not fork the repo though)



Within-workflow file structures

- Within each of the files in our workflow, it is important to maintain a clear and consistent file organizational structure
- This means adding folders as needed and clearly numbering each R script and labeling Quarto files, PDFs, and other documents part of the workflow (like we saw on the previous slide)
- Each of these scripts and files should have a clearly written preamble that establishes who wrote them, their main goal, and any additional information (such as prereqs) to help with reproducibility and overall organization

R script preambles

- We include a preamble at the start of our R scripts to outline:
  - the purpose of the document;
  - the author and contact details;
  - when the file was written or last updated
  - prerequisites that the file relies on
- In R, lines that start with "#" are comments, meaning that they are designed to be read by humans and are *not* run as code by R

Example: R script preamble

- Here's an example preamble from a paper I am currently working on

```
#### Preamble ####
# Purpose: Clean candidate and Bluesky account data
# Author: Inessa De Angelis
# Date: 20 February 2026
# Contact: inessa.deangelis@mail.utoronto.ca
# License: MIT
# Pre-requisites:
  # Run 01-download_data.R first
```

- Stylistic variations of this preamble are fine, as long as the basic information is covered

GitHub

- GitHub is a platform for hosting and managing version-controlled projects using Git
- It allows you to:
  - Track changes to your code and documents over time
  - Collaborate with others on shared projects
  - Back up your work in the cloud

Key GitHub Terms

- **Repository (repo):** your project folder
- **Commit:** saving changes to your files and maintaining a record of the modifications
- **Push:** uploading changes to GitHub
- **Pull:** downloading updates from GitHub

What happens when our workflows do not use GitHub?

- Files can get overwritten or lost
- You or other scholars cannot reproduce your results later on
- File management becomes confusing for you and/or your collaborators (avoids having files called "`final_final_paper_v2.qmd`")

How everything fits together

- **RStudio (local):** where you write and run your code
- **Git:** tracks changes to your files
- **GitHub (remote):** stores and shares your project

Getting a Personal Access Token (PAT)

- A **Personal Access Token (PAT)** is used to authenticate with GitHub instead of your password
- In RStudio, it allows you to securely push, pull, and interact with GitHub

Steps to create a PAT

- Go to GitHub -> Settings -> Developer settings -> Personal access tokens
- Select **"Tokens (classic)"** -> "Generate new token"
- Give your token a name (for example, "RStudio access")
- Set an expiration date (**August 15th, 2026 or later**)
- Select scopes: `repo` and `workflow` (full control of repositories)

Important note!

- Copy and save your token immediately (you won't be able to see it again)
- Store it securely (for example, in your `.Renviron` file using `GITHUB_PAT = ...`)
- Never share or commit your token to GitHub!

Linking RStudio and GitHub

- Let's check together that our RStudio and GitHub are linked - please follow along
- If you have any issues, please consult Chapter 3 of *Telling Stories with Data* and Chapters 12 and 13 of *Happy Git and GitHub for the useR*
- If you are still having issues, please ask someone sitting near you first and then you can ask me during the break/lab if needed

From manual to automated workflows

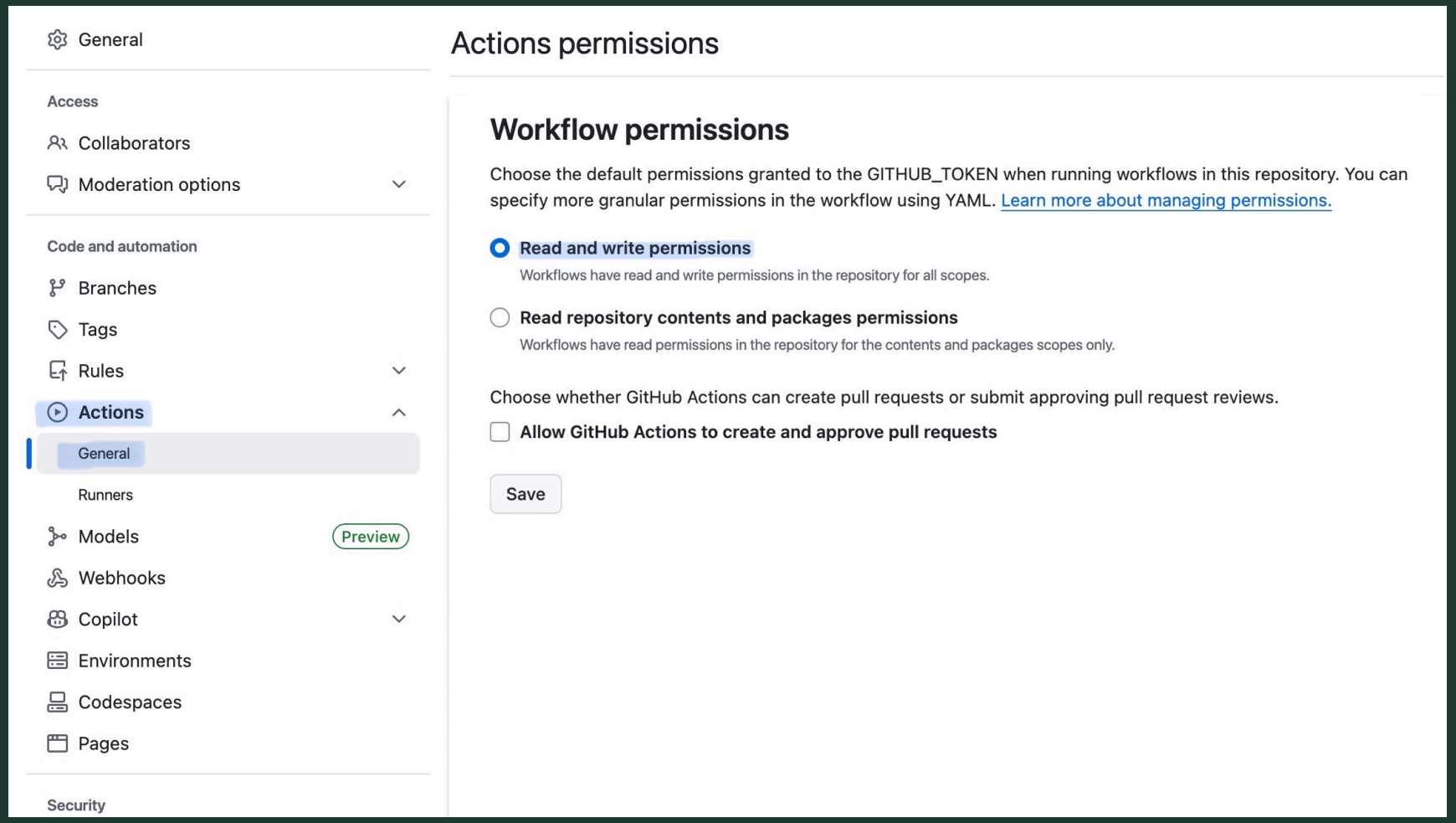
- So far: We've thought about designing research workflows manually
- Next step: We can automate parts of these workflows using tools like GitHub Actions

Using GitHub Actions to automate workflows

- There are several tools for automating research workflows, such as Docker, GitLab CI/CD, and GitHub Actions that help document each stage of a project and enable continuous integration and deployment
- We're going to focus on GitHub Actions because it integrates directly with GitHub
- Automating workflows can increase transparency by clearly documenting every step in the data collection and processing pipeline
- It can also improve reliability: instead of manually collecting data, workflows can run on a schedule, reducing the risk of missing data during high-intensity periods (for example, election campaigns)

Getting Started with GitHub Actions

- First enable “read and write permissions” so your GitHub Token has the correct permissions to execute workflows and commit files

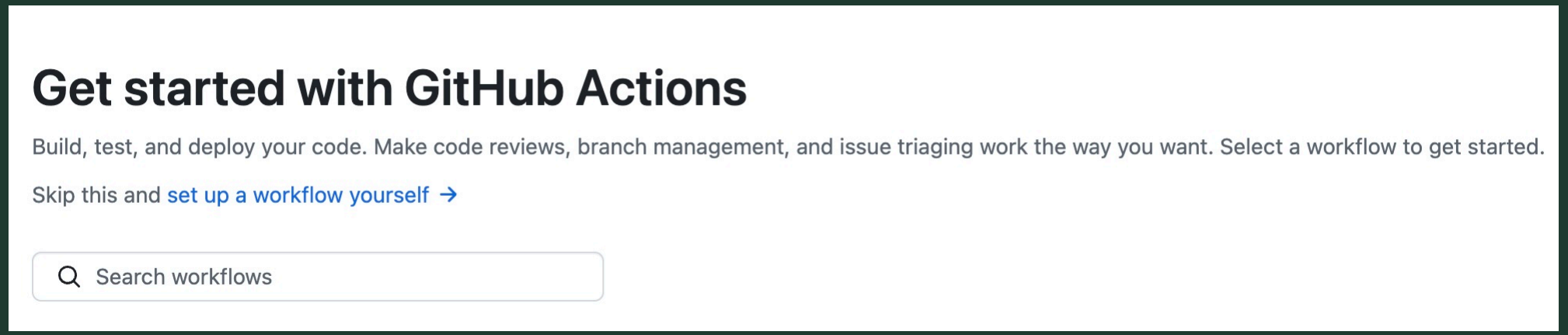


Getting Started with GitHub Actions (continued)

- Add your sensitive information (usernames, keys, and passwords) required to run your workflow to the “secrets” setting of your repository

Setting up your Workflow

- We can now set up our ".YAML" file either through the GitHub interface or locally in your code editor: "*.github/workflows/*"



A quick word on “YAML”

- Workflows must be written using the “YAML” syntax, which is a human friendly data serialization standard for all programming languages
- YAML syntax is similar to XML and JSON, but requires indentations like Python
- There is no functional difference between files with a “.YML” or “.YAML” extension, however “.YAML” is the newer and preferred version
- Your YAML file must include the following core components in order to successfully run: **Events, Jobs, Runners, Steps,** and **Actions**.

Elements of the Workflow: Events

- The “on” field specifies the event(s) that trigger the workflow (such as ‘*push*’, ‘*pull_request*’, or ‘*schedule*’)
- For workflows designed to automate daily or weekly social media data collection, the schedule trigger is used in conjunction with a cron expression (which is in Coordinated Universal Time - UTC)

```yaml
on:
  schedule:
    - cron: '0 7 * * *' # run every day at 7am UTC
```

Elements of the Workflow: Jobs

- '*Jobs*' defines the core tasks that will be executed as part of the workflow
- '*Collect_posts*' is the unique identifier or job ID that each job is assigned
- '*Name*' is the title for your workflow in the "Actions" tab of your repository
- Each job is self-contained within a specific virtual environment called the "*runner*" and you specify this using the '*runs-on*' command

```
jobs:
  collect_posts:
    name: Collect CdnPoli posts from Bluesky
    runs-on: macOS-latest
```

Elements of the Workflow: Steps

- Each job in a GitHub Actions workflow is comprised of a series of steps, which are indicated using bullet points and need to be executed in order
- Steps may use predefined GitHub Actions (indicated by “uses”) or custom commands (indicated by “run”)
- Here, the runner clones our repo, installs R and the relevant R packages

```
steps:
  - uses: actions/checkout@v2

  - uses: r-lib/actions/setup-r@v2

  - name: Install dependencies
    run: Rscript -e 'install.packages(c("tidyverse", "atrrr"))'
```

Elements of the Workflow: Steps

- This step accesses the secret environment variables (Bluesky username and password) from our repository settings
- It also runs our R script (01-download_Bluesky_data.R) to collect the latest '#CdnPoli' posts and save them in a CSV file

```
    - name: Collect CdnPoli posts from Bluesky
      env:
        BLUESKY_USERNAME: ${{ secrets.BLUESKY_USERNAME }}
        BLUESKY_AUTH: ${{ secrets.BLUESKY_AUTH }}
      run: |
        Rscript "01-download_Bluesky_data.R"
```

Elements of the Workflow: Steps

- Now we configure our Git user credentials using the secret repo variables we previously saved to commit our new/updated files to our repo
- GitHub Actions then stages all changes and commits them to our repo or prints the message “no changes to commit”

```
- name: Configure git user
  run: |
    git config --global user.name "${{ secrets.GIT_USERNAME }}"
    git config --global user.email "${{ secrets.GIT_EMAIL }}"

- name: Commit results
  run: |
    git add -A
    git commit -m 'New data!' || echo "No changes to commit"
    git push origin || echo "No changes to commit"
```

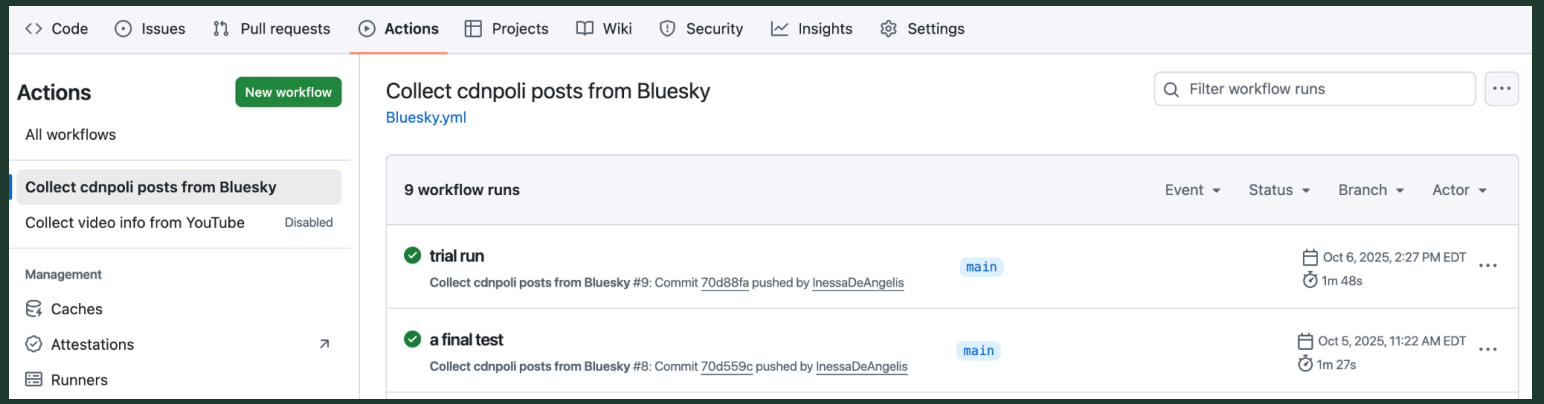
Elements of the Workflow: Clean up

- This step is not specified in our YAML workflow file, but GitHub Actions automatically concludes the workflow by performing behind-the-scenes cleanup tasks (for example, removing temporary credentials)
- We only see this step in our workflow log

```
Post Run actions/checkout@v2
```

Review your Workflow Log!

- After your workflow has run, you can review the detailed execution logs in the “Actions” tab of your repository to verify success or identify where there were errors
- Workflow logs are retained for up to 90 days by default
- Reviewing this information is essential for debugging and maintaining a reproducible workflow
- It is good practice to regularly review your datasets collected via GitHub Actions to ensure that there is no missing or incorrect data



Live demonstration: Bluesky data collection

- To see everything come together, let's collect some Bluesky posts tagged with the popular Canadian political hashtag '#CdnPoli'
- Follow along: https://github.com/InessaDeAngelis/GitHub_Actions_PolComm

Concluding Thoughts on GitHub Actions

- Automating your workflows can help you think through the steps of your research process
- Using GitHub Actions can embed reproducibility into the research process from the outset because it forces researchers to proactively document all steps in their workflows
- GitHub Actions has many potential uses beyond data collection
- Can contribute to the overall reproducibility and trustworthiness of projects

Survey

- Go follow this link to go to the class survey on Microsoft Forms: https://forms.cloud.microsoft/r/zFFbbGRRfx
- Please fill out the survey to help me learn about you so that I can more effectively teach this course

Your turn

- Go to the class GitHub repository, https://github.com/inessadeangelis/INF2167 and to the “Week 2” folder
- Download the lab questions called `week2_lab.qmd` that are in that folder and open them in your RStudio
- We will start the lab by working individually
- When we move into the group part of the lab (I’ll tell you when), please work with your assigned partner for the group project

Submission

- Please email your .qmd file and PDF to Inessa.DeAngelis@mail.utoronto.ca with the subject line “INF2167: In-class Lab 2 Submission” **OR**
- Upload your .qmd file and PDF to Quercus under “Assignments” -> “In-class Lab 2”

For next week

- *Telling Stories*, Chapters 5 & 9
- *Modern Dive*, Chapters 2 & 3
- D'Ignazio, C., & Klein, L. (2020). 3. "On Rational, Scientific, Objective Viewpoints from Mythical, Imaginary, Impossible Standpoints." In *Data Feminism*. https://data-feminism.mitpress.mit.edu/pub/5evfe9yd/release/5
- Vanderplas, S., Cook, D., & Hofmann, H. (2020). Testing Statistical Charts: What Makes a Good Graph? *Annual Review of Statistics and Its Application*, *7*(1), 61–88. https://doi.org/10.1146/annurev-statistics-031219-041252

References

Alexander, R. (2023). *Telling Stories with Data: With Applications in R*. Chapman & Hall/CRC.

De Angelis, I. (2024). Torrential Twitter? Measuring the Severity of Harassment When Canadian Female Politicians Tweet About Climate Change. *Social Media + Society*, *10*(4), 1–17. https://doi.org/10.1177/20563051241304493

De Angelis, I. (2026). Using GitHub Actions for Computational Communication Research. *SocArXiv*. https://doi.org/10.31235/osf.io/uqf6n_v1

Appendix: Installing Git for Windows

If Git is not automatically installed, please follow these steps

- Install Git for Windows, also known as `msysgit` or "Git Bash," to get Git
- More detailed instructions are in Chapter 6 of *Happy Git and GitHub for the useR*